

# Classification Model for the Evaluation of Holstein Sires

Bárbara Paola Alcántara Vega

Professor: Dr. Benjamín Valdés Aguirre

**Abstract**—The main objective of this report is to break down the analysis behind the model developed to classify levels of acceptance for 17 linear conformation characteristics in bulls within the *SelectSires* database for the Holstein sire breed. The proposed model includes current limitations and proposals for improvement, since the model currently has an average accuracy of 85.5% for the training and testing of each of the classes.

**Keywords**—sire, gradient descent, logistic regression, XGBoost, features, classes

## 1. Introduction

Sire semen selection is a common practice of dairy farms to ensure progeny with characteristics that maximize productivity while maintaining the health of the herd. This is the reason why companies have developed an extensive catalog with detailed data on the physical and genetic characteristics of each bull they offer, in order to give producers a selection of the bulls that most benefit production based on characteristics already present within the dairy herd. However, many producers don't have the time or knowledge to do an extensive analysis of the characteristics of the bulls they could choose, and end up choosing bulls that, although good, do not maximize the productive performance of the herd, and in the worst case, if only one or two bulls are chosen for the entire herd, the herd ends up with consanguinity problems and further expenses.

Normally, the evaluation and selection of livestock requires a certified expert for the breed that it's intended to work with, but with recent advances in machine learning, it is possible to automate the evaluation process not only through scoring, but also with image analysis, allowing producers to carry out an appropriate evaluation of their livestock and their selection of bulls without having previous experience of the breed in terms of linear conformation traits.

In this context, bull selection with machine learning allows bulls to be classified based on their score; very low, low, high and very high. Considering various attributes such as designations and guidelines to adapt to the needs of each producer; such as optimization in the use of forage feed, grass feeding, gender selection for progeny, and more; the applications of Artificial Intelligence and machine learning are becoming increasingly accessible, allowing producers to make informed and competitive decisions more easily. Not only allowing better selection of genetic material, but also an insight to the factors that best suit them, improving their dairy herd in terms of production and health.

## 2. Dataset's description

The bull sire database, which serves as the basis of the project, is derived from public access to the sire catalog containing characteristics organized and classified by the company *SelectSires*, for the selection and identification of its sires for the buyer and even for decision making when it comes to getting new genetics for the consumer. They handle a variety of breeds, including the Holstein breed, which is the focus of this project. The reason this company was selected for acquiring the data base is that this database is one of the few open source data available on the internet that contains the linear traits for the Holstein breed as specified in the Holstein Association USA, Inc. Linear Trait Manual, which is the standard in cattle grading and classification. The database relates directly to the attributes that qualify a sire for different needs.

## 3. Features

The data set includes 161 columns, each representing an important characteristic of the promoted sire. However, in order to simplify

the analysis, 17 main characteristics were selected as to universally evaluate livestock and their main identifiers:

### 3.1. Main identifiers

- **NAAB**: Serial number for the stallion
- **Lineups/designations**: Class to which the stallion belongs

### 3.2. Main traits

- **PTAT**: Body depth
- **STA**: Stature
- **STR**: Strength
- **DFM**: Dairy form
- **RUA**: Rump angle
- **RLS**: Rear legs from lateral view
- **RTP**: Rear teat placement
- **FTL**: Front teat length
- **RW**: Rump width
- **RLR**: Rear legs from rear view
- **FTA**: Hoof angle
- **FUA**: Front udder insertion
- **RUH**: Rear udder height
- **RUW**: Rear udder width
- **UCL**: Udder cleft ligament
- **UDP**: Udder depth
- **FTP**: Front teat placement

Additionally, it is important to take into account that each characteristic has a percentage of heritability and a level of importance in average health and milk production.

Fig. 1 shows the percentages mainly considered in linear traits for milk production.



**Figure 1.** Relevance of each group of traits in health and milk yield (Holstein Canada: Services - Classification, 2015).

## 4. Data cleaning

The database contains 161 features, and not all the data they contain is numerical. So the first thing that had to be done is to swap the columns that did not provide relevant or clear information in order to work with numerical data that can be used in learning and training the model. Furthermore, although the database instances are already found randomly, randomization is necessary between each training and test so that the model does not incorrectly fit the data and can predict scores regardless of the order of the data. data provided.

Equation 1 shows how the data normalization was carried out:

$$X_{norm} = \frac{X - \min(X)}{\max(X) - \min(X)} \quad (1)$$

This way, we keep the range of each feature on a standard scale, since each feature has data whose range is variable with respect to

the others. This numerical stability ensures that each characteristic is within a comparable scale and thus prevents higher values from dominating the training process.

## 5. Correlation analysis and feature selection

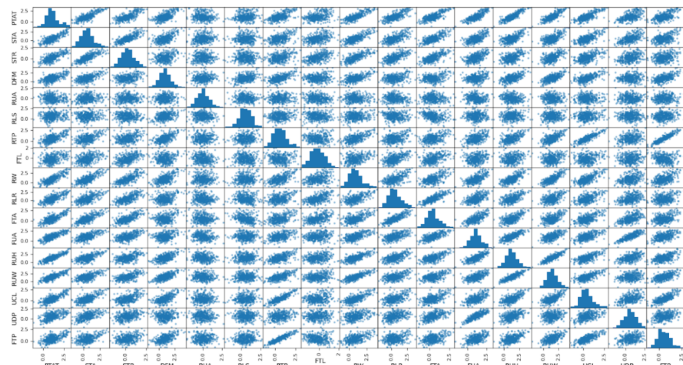


Figure 2. Scatter plot depicting the 17 selected features.

Thanks to a scatterplot of each linear conformation trait, we can observe that the relationship between data involved with udders and legs are closely related, this relation makes sense as these are two dependent characteristics for the development of good breast health in the Holstein breed. This is why farmers tend to focus on these characteristics and tend to be misguided by data such as the one showed on figure 1, however, although it helps a lot in terms of productivity, only focusing on udders and legs could result in other problems, such as lack of strength in the legs and accidental omission of the suspensory tendon, two very important characteristics for the health of the animals as individuals and a greater number of productive days. The characteristics describing ppm solids in milk, pedigree, or dollars per liter of milk on average were not maintained, since in addition to the fact that the Holstein breed is well known as a producer of fluid milk and not of dairy products, the focus on health and productivity is a priority in improving current dairy production nationwide in Mexico.

## 6. Data set split

Dividing the database into three parts, allows training, validation and testing of the results. For this to work, it is important that the proportions of this division are 60%, 20% and 20% respectively for each part. All this with the aim of training the model with the greatest amount of data available possible and consequently, evaluating the model with respect to its performance as if new data were presented, thus simulating a selection in a real scenario.

## 7. Base model

The project uses a linear regression model built from its basic elements without using existing machine learning libraries, this with the aim of deepening the understanding of the functioning of these processes during data analysis. This model is mainly used as a binary selection tool, but can also be used for multi-class classification. Due to the number of classes that need to be predicted, the cycle must be repeated for each of the 12 classes. In each cycle, the model uses the strategy called One-vs-Rest.

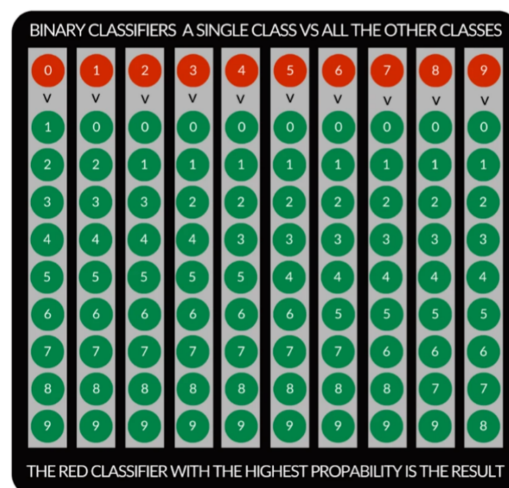


Figure 3. Basic One-vs-Rest depiction (Psaltakis et al., 2024).

This approach involves training a binary classifier for each class, where the classifier of a specific class is trained to distinguish that class from all other classes combined [(OneVsRestClassifier, 2025)]. This way, instead of trying to predict the 12 classes at the same time, it focuses on a single class to treat the rest of the 11 as a class by itself, with the aim of determining whether the data belongs to that class by applying the hypothesis function, and only then, after having repeated the process for three classes, it finds the index of the class with the highest probability of corresponding to the analyzed data.

## 8. Hypothesis function

The hypothesis function is used to predict the probability that an input data from the data belongs to a specific class or combination of classes.

Equation 2 shows the sigmoid function used:

$$h\theta = \frac{1}{1 + e^{-\theta^T x}} \quad (2)$$

Because its domain remains in all real numbers, and its range is (0,1), even if a large positive or negative number is received, the result will always be between 0 and 1. It is for this same reason that the One-vs-Rest method is suitable for selection, since it can only determine between 0 and 1.

## 9. Cost function

The cost function quantifies the difference between the predicted probabilities and the current class to which they belong, so if we minimize this difference during training, we allow the model to fit its parameters to improve certainty. Since we have 12 classes, the cost function is used for each of them independently.

Equation 3, shows the cost function used:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[ y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] \quad (3)$$

## 10. Gradient descent

The gradient descent allows the model to learn from the data by minimizing the cost function. Which means that the gradient of the cost function with respect to the parameters (theta) is calculated first, and subsequently, each parameter is updated in the direction that reduces the cost, thus increasing the accuracy of the predictions.

$$\Theta_j := \Theta_j - \frac{\alpha}{m} \sum_{i=1}^m \left( (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} \right) \quad (4)$$

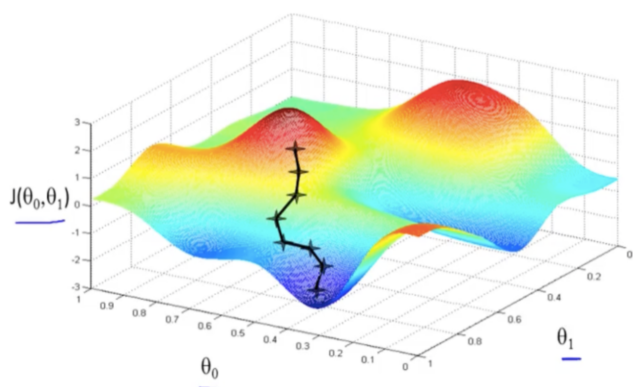


Figure 4. Illustrative depiction of gradient descent [(DBCerigo, 2018)].

### 10.1. Why not use back-propagation?

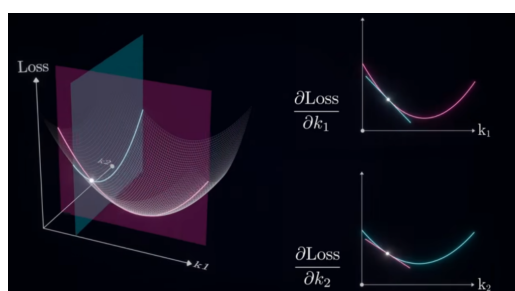


Figure 5. Illustrative depiction of back-propagation [(Artem Kirsanov, 2024)].

As it is shown further on, the current model works well with a single perceptron and logistic regression, and thus a more complex model with a neural network could benefit much more from back propagation than this demonstration of theoretical and practical knowledge, as the derivatives slicing each loss in the three dimensional curve would be more heavy to process than what a simple vector with updating parameters (*theta*) for the same result.

## 11. Training, validation and testing

The model learns by adjusting parameters using gradient descent, minimizing the error in the predictions for each epoch. The One-vs-Rest strategy is used to train the model separately for each class, while the strategy is monitored to observe progress.

In addition, the behavior and progress of the model is also evaluated with a separate validation set of training data. This is with the aim of recording how well the model generalizes to data never seen before. In this way, monitoring validation certainty allows the model to be adjusted while preventing overfitting during training.

After training, the model is tested with completely new data (the test set), to evaluate its generalization ability, while the final certainty test tells us how likely it is that the model behaves correctly in a real scenario.

## 12. Base model results

After training the model and testing it, the results show that the model achieved an average certainty of 85% in training. Furthermore, a reduction in error can be observed by half after 300 epochs.

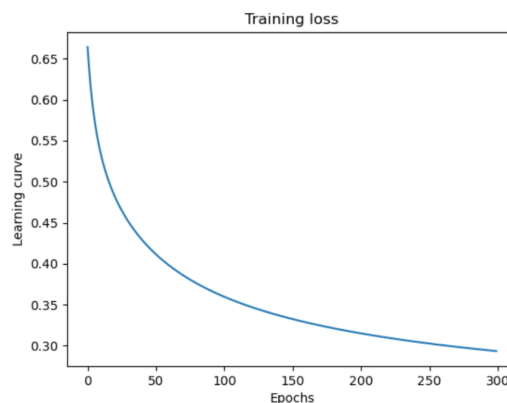


Figure 6. Mean loss curve.

The error starts relatively high around 0.66, and as the epochs increase, the loss drops to stabilize around 0.29, showing that the model is successfully minimizing the error. Since the curve does not have oscillations or sharp increases, the model learns and reduces its error with each iteration without presenting obvious signs of overfitting (without increase in loss/error), in addition, the convergence is progressive with a marginal improvement around the 250-300 epochs, which means that training for more epochs would provide minimal improvement in error.

Table 15 Regarding the certainty results for each of the classes:

Table 1. Each feature's accuracy

| Feature              | Accuracy     |
|----------------------|--------------|
| PTAT                 | 0.898        |
| STA                  | 0.874        |
| STR                  | 0.882        |
| DFM                  | 0.840        |
| RUA                  | 0.715        |
| RLS                  | 0.785        |
| RTP                  | 0.914        |
| FTL                  | 0.777        |
| RW                   | 0.835        |
| RLR                  | 0.825        |
| FTA                  | 0.869        |
| FUA                  | 0.898        |
| RUH                  | 0.869        |
| RUW                  | 0.887        |
| UCL                  | 0.882        |
| UDP                  | 0.911        |
| FTP                  | 0.882        |
| <b>Mean accuracy</b> | <b>0.855</b> |

Please note that FTL has a low level of accuracy.

The training graph, together with our observed accuracies table, indicate that there is no presence of overfitting. Fortunately, none of the characteristics exceed 95%, which would indicate a suspicious level of accuracy, although it is important to note that the front teat length (FTL) does not have a very high level of accuracy. This makes sense, since nipple length does not usually influence classes, except for *RobotPRO*, where nipple length facilitate but does not determine automatic milking, meaning that having long or short nipples does not completely prevent automatic milking.

The model seems to work without signs of overfitting using 300 epochs, where the training process looks quite successful with a validation certainty level of approximately 91%, and a validation loss of 0.3, value from which, there is no longer a significant difference in the decrease in error.

The following figure shows the comparison between Training Loss/Training Accuracy and Validation Loss/Validation Accuracy.

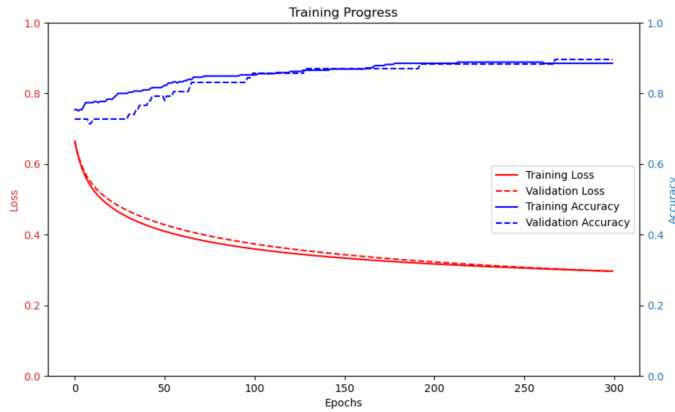


Figure 7. Graph with training progress for validation and loss.

The model learns effectively from training data while generalizing well to new data not seen before (validation set). So there are no signs of overfitting or severe underfitting. Furthermore, the certainty measure shows us that 91% of the time a correct classification is being made for the classes.

The smooth and constant curve shows us that the parameters for the “learning rate” are appropriate, since a very high learning rate can cause disturbances in the curve, and a very low learning rate can cause a much slower convergence.

The following figure shows the confusion matrix regarding Misclassification, True Positive Rate, False Positive Rate and True Negative Rate for the gradient descent+ logistic regression as a binary classification model.

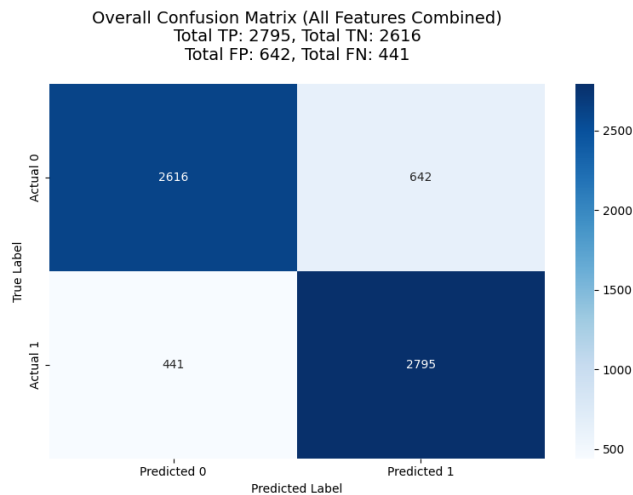


Figure 8. Confusion matrix for the gradient descent + logistic regression.

The confusion matrix shows great performance, having low misclassification and maintaining 81.28% precision, however, when looking at the bias and variance scores, we can see evidence of overfitting, as the bias is low and variance is high.

Table 2. Each feature's bias and variance

| Feature  | Bias           | Variance          |
|----------|----------------|-------------------|
| PTAT     | 0.110          | 0.247             |
| STA      | 0.094          | 0.248             |
| STR      | 0.042          | 0.250             |
| DFM      | 0.147          | 0.245             |
| RUA      | 0.087          | 0.248             |
| RLS      | 0.026          | 0.250             |
| RTP      | 0.014          | 0.250             |
| FTL      | 0.065          | 0.249             |
| RW       | 0.120          | 0.246             |
| RLR      | 0.013          | 0.250             |
| FTA      | 0.058          | 0.249             |
| FUA      | 0.144          | 0.245             |
| RUH      | 0.137          | 0.246             |
| RUW      | 0.086          | 0.248             |
| UCL      | 0.099          | 0.248             |
| UDP      | 0.063          | 0.249             |
| FTP      | 0.058          | 0.249             |
| Average: | Bias<br>0.0805 | Variance<br>0.248 |

### 12.1. Model improvement

After obtaining these results, the model underwent some attempts to reduce the minimal overfitting it was presenting, but attempts in reducing overfitting only resulted in an increase in bias, no change in variance and a very slight increase in overfitting.

The methods for reducing overfitting were the following:

### 12.2. Lasso Regression (L1) and Ridge Regression (L2) regularization

In L2 regularization, the regression coefficients are obtained by minimizing the L2 loss function, given as:

$$J(w)L_2 = \sum_i^n (y^{(i)} - \hat{y}^{(i)})^2 + \alpha \|w\|_2^2 \quad (5)$$

where

$$\|w\|_2^2 = \sum_j^m w_j^2 \quad (6)$$

The main difference with L1 regularization is that the regression coefficients are obtained by minimizing the L1 loss function, which does not have Euclidean norm squared of the weight vector, and is given as:

$$J(w)L_1 = \sum_i^n (y^{(i)} - \hat{y}^{(i)})^2 + \alpha \|w\|_1 \quad (7)$$

where

$$\|w\|_1 = \sum_j^m |w_j| \quad (8)$$

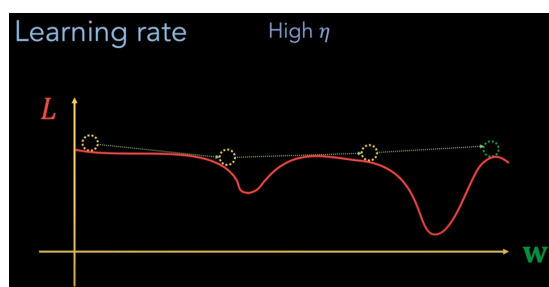
The L1 regularization only the component vector of the weight, while L2 regularization contributes quadratically to the norm (larger weights are penalized more heavily because squaring magnifies them).

So in order to tune our model towards less overfitting, L2 regularization serves as a larger modification and once we get to a close desired value, L1 serves as a more subtle weight to set everything in place (Tayo, 2022).

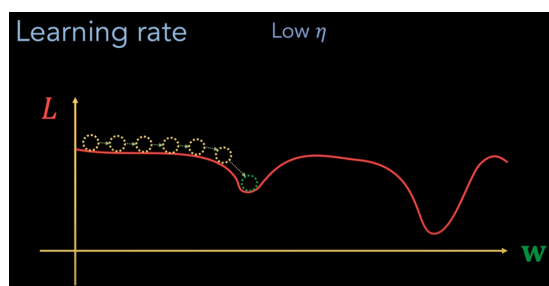
### 12.3. Reducing learning rate and adding early stopping

Reducing the learning rate during training improves performance by allowing the model to make finer and more precise adjustments to weights as it approaches to the minimum of the loss function. This means that the larger a learning rate is set, the convergence is more unstable (Brownlee, 2019).





**Figure 9.** Graphic example of a learning rate tuned too high (DeepBean, 2023).



**Figure 10.** Graphic example of a learning rate tuned too low (DeepBean, 2023).

Reducing the number of epochs can also lower overfitting scores, as the more repetitions are done, the more the model “memorizes” the training set and be worse at classifying or predicting new data. The simplest way of limiting the number of epochs is by using early stopping based on the validation scores, stopping the model training when it’s performance plateaus or begins to degrade to a messy curve (Brownlee, 2019).

#### 12.4. Cross-validation

Cross-validation is a statistical technique that helps to assess how the results of a prediction model will generalize to an independent data set by partitioning the available data into complementary subsets; one for training the model and another for validating its performance. There are two categories of cross-validation methods, exhaustive and non-exhaustive. In this particular case, k-folding was used, which is a non-exhaustive method consisting of randomly dividing data into k equal-sized subsets or “folds”. Using k-1 folds as a training set and the remaining folds as validation sets, repeating each process k times (k can be any positive integer) which each fold used exactly one per each iteration of the validation set. Then, the performance metric (accuracy and/or mean squared error) from each fold is averaged to produce a single, more robust estimate of the model’s predictive performance (GeeksforGeeks, 2025). This way, a more accurate and less optimistic estimate of the model’s performance is achieved in order to stimulate generalization to new data.

#### 12.5. Results after overfitting reduction methods were applied:

**Table 3.** Original model’s final training progress values.

| Feature             | Accuracy |
|---------------------|----------|
| Testing loss        | 0.2967   |
| Validation loss     | 0.2958   |
| Testing accuracy    | 0.8852   |
| Validation accuracy | 0.8961   |

**Table 4.** Model’s final training progress values when applying overfitting reduction methods.

| Feature             | Accuracy |
|---------------------|----------|
| Testing loss        | 0.2981   |
| Validation loss     | 0.2977   |
| Testing accuracy    | 0.8852   |
| Validation accuracy | 0.8961   |

**Table 5.** Model’s final bias and variance before and after applying overfitting reduction methods.

| Bias (before) | Var. (before) | Bias (after) | Var. (after) |
|---------------|---------------|--------------|--------------|
| 0.080         | 0.248         | 0.085        | 0.248        |

Variance being around 0.2 is common for binary classification at its first (GeeksforGeeks, 2025), and this model looks to be working very well under an 85.5% accuracy, and 81% precision. This might be the reason why reducing overfitting is not working with conventional overfitting reduction methods and the reason why changing the curve equation might have more success in making bias and variance comparable.

An overall summary is shown below:

**Table 6.** Evaluation metrics.

| Metric                            | scores |
|-----------------------------------|--------|
| Total true positives (TP):        | 2795   |
| Total true negatives (TN):        | 2616   |
| Total false positives (FP):       | 642    |
| Total false negatives (FN):       | 441    |
| Overall bias score (Average):     | 0.080  |
| Overall variance score (Average): | 0.248  |

**Table 7.** Top 5 best performing models by accuracy score.

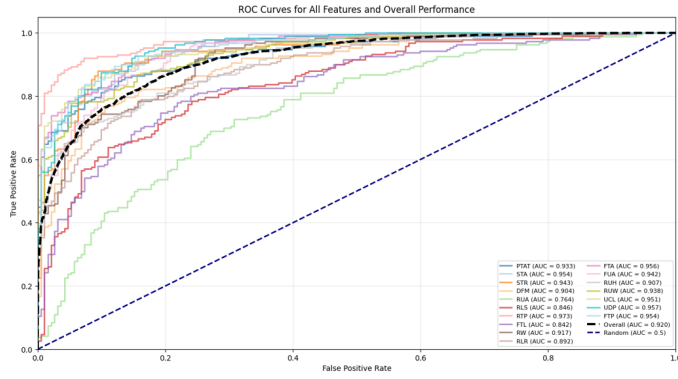
| No. | Feature | Accuracy | Bias | Precision |
|-----|---------|----------|------|-----------|
| 1.  | RTP:    | 91.4%    | 1.4% | 90.6%     |
| 2.  | UDP:    | 88.5%    | 6.3% | 86.2%     |
| 3.  | STR:    | 88.5%    | 4.2% | 86.9%     |
| 4.  | FTP:    | 87.7%    | 5.8% | 85.6%     |
| 5.  | UCL:    | 87.7%    | 9.9% | 84.3%     |

**Table 8.** Top 5 best performing models by bias score.

| No. | Feature | Accuracy | Bias | Precision |
|-----|---------|----------|------|-----------|
| 1.  | RRL:    | 79.6%    | 1.3% | 79.8%     |
| 2.  | RTP:    | 91.4%    | 1.4% | 90.6%     |
| 3.  | RLS:    | 77.2%    | 2.6% | 76.5%     |
| 4.  | STR:    | 88.5%    | 4.2% | 86.9%     |
| 5.  | FTP:    | 87.7%    | 5.8% | 85.6%     |

Fortunately, production wise, the best performing features are mainly related with udder health, leg health and strength and milk yield, securing a good selection for mammary health and gland performance along with leg health and strength. However, traits related to rump width, and therefore calving ease, don’t have the best accuracy or bias scores, so when selecting a bull or classifying one with this

model, it is advised to procure a wider ramp in order to promote calving ease in the daughters resulting from the breeding program.



**Figure 11.** ROC curve for the Logistic Regression + Gradient Descent classification model.

There are some excellent performers; UCR (AUC = 0.953), RTP (AUC = 0.929), TFP (AUC = 0.936), good performers; RES (AUC = 0.866), ITT (AUC = 0.842), moderate performers; RUA (AUC = 0.764), and the rest, with a near random AUC of 0.5-0.6.

Being that the majority of the features are near random AUC, It's evident that adding more information to the data set could greatly benefit the AUC scores, given that many classes contain a small amount of sires, as it seems the overfitting reduction methods that were applied did not reduce this variability. The overall AUC of 0.920 indicates the Logistic Regression + Gradient Descent model performs very well when considering all features combined. The features UCR, RTP and TFP have excellent discriminatory power, distinguishing between classes with minimal problems.

### 13. Comparing models: Gradient descent + logistic regression V.S. XGBoost

Additionally, the model was compared with another model made with XGBoost and scikit-learn to see if Logistic Regression + Gradient Descent was enough, or if a more complex framework was needed.

#### 13.1. Extreme Gradient Boosting (XGBoost)

XGBoost handles tabular time series data exceptionally well, building trees like a Random Forest but sequentially. Each new tree corrects the error of the previous one, making the model capable of learning from mistakes and focusing on hard to predict examples. This model is exceptional for structures/tabular data, making it very accurate, but slower to train, as it's sequentiality while making the trees can make it prone to overfitting if not tuned properly (GeeksforGeeks, 2025).

Mathematically the model can be represented as:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i) \quad (9)$$

Where:

- $\hat{y}_i$  is the final predicted value for the  $i^{th}$  data point
- $K$  is the number of trees in the ensemble
- $f_k(x_i)$  represents the prediction of the  $k^{th}$  tree for the  $i^{th}$  data point

Now, the objective function in XGBoost consists of two parts: a loss function and a regularization term. The loss function measures how well the model fits the data and the regularization term simplify complex trees (GeeksforGeeks, 2025). The general form of the loss function is:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i) \quad (10)$$

Where:

- $\hat{y}_i^{(t-1)}$  is the prediction from the previous iteration
- $f_t(x_i)$  is the prediction of the  $t^{th}$  tree for the  $i^{th}$  data point

The regularization term  $\hat{\Omega}(f_t)$  simplify complex trees by penalizing the number of leaves in the tree and the size of the leaf (GeeksforGeeks, 2025). It is defined as:

$$\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (11)$$

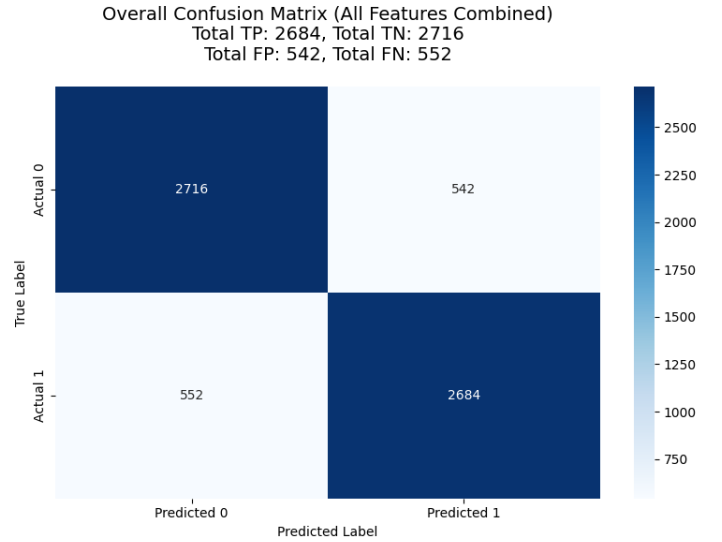
Where:

- $T$  is the number of leaves in the tree
- $\gamma$  is a regularization parameter that controls the complexity of the tree
- $\lambda$  is a parameter that penalizes the squared weight of the leaves  $w_j$

#### 13.2. Why not use Random Forest?

Random forest does not have an inherent way to save the metrics we need to compare with our logistic regression + gradient descent model. Even though we do get Misclassification, True Positive Rate, False Positive Rate and True Negative Rate, it is difficult to store Training Loss/Training Accuracy and Validation Loss/Validation Accuracy, as loss is not inherently calculated and therefore, we can't compare models with all the metrics used for diagnostic without adding more libraries and frameworks such as XGBoost.

The following figure shows the confusion matrix regarding Misclassification, True Positive Rate, False Positive Rate and True Negative Rate for the XGBoost as a binary classification model.



**Figure 12.** Confusion matrix for the XGBoost framework.

The confusion matrix shows good performance, having low misclassification rate and maintaining 96.48% precision, however, when looking at the bias and variance scores -similar to the Logistic Regression + Gradient Descent case- we see evidence of overfitting, as the bias is low and variance is high. This is also explained with the typical value of 0.2 for binary classification models. Either way, overfitting reduction methods were implemented in order to see if it was possible to reduce variance without touching the curve's equation.

Furthermore, it is important to consider that XGBoost tends to overfit, something evidently seen in the table below:

**Table 9.** Overall metrics for XGBoost model.

|                            |        |
|----------------------------|--------|
| Final Training Loss:       | 0.1814 |
| Final Validation Loss:     | 0.3765 |
| Final Training Accuracy:   | 0.9859 |
| Final Validation Accuracy: | 0.8312 |

Apart from tuning the basic parameters to have a smaller learning rate (0.05), samples and sub samples, in order to reduce overfitting, L1 and L2 regularization was used.

A great advantage of using the *XGBoost* library over a manual execution on both linear regression and gradient descent, is that changing parameters for tuning is far more easier by visualization, allowing us to clearly see the set weights for each regression equation.

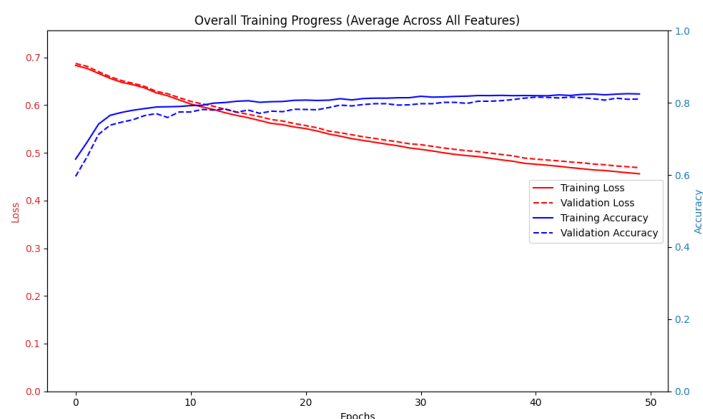
```

1  params = {
2      'objective': 'binary:logistic',
3      'eval_metric': ['logloss', 'error'],
4      'max_depth': 3,
5      'learning_rate': learning_rate,
6      'subsample': 0.07,
7      'colsample_bytree': 0.7,
8
9      'reg_alpha': 0.5, # L1 regularization
10     'reg_lambda': 1.3, # L2 regularization
11
12     'random_state': 42
13 }
14

```

**Code 1.** Parameter setting section on Python code.

Greatly reducing overfitting:

**Figure 13.** Average training progress across all features.

And ending with the following bias and variance:

**Table 10.** Each feature's bias and variance for XGBoost.

| Feature         | Bias                  | Variance                 |
|-----------------|-----------------------|--------------------------|
| PTAT            | 0.010                 | 0.250                    |
| STA             | 0.031                 | 0.250                    |
| STR             | 0.042                 | 0.250                    |
| DFM             | 0.099                 | 0.248                    |
| RUA             | 0.129                 | 0.246                    |
| RLS             | 0.162                 | 0.242                    |
| RTP             | 0.019                 | 0.250                    |
| FTL             | 0.024                 | 0.250                    |
| RW              | 0.047                 | 0.249                    |
| RLR             | 0.107                 | 0.247                    |
| FTA             | 0.120                 | 0.246                    |
| FUA             | 0.082                 | 0.248                    |
| RUH             | 0.038                 | 0.250                    |
| RUW             | 0.039                 | 0.250                    |
| UCL             | 0.010                 | 0.250                    |
| UDP             | 0.005                 | 0.250                    |
| FTP             | 0.037                 | 0.250                    |
| <b>Average:</b> | <b>Bias</b><br>0.0151 | <b>Variance</b><br>0.249 |

As the variance was not reduced by the typical overfitting reduction methods, the only way to reduce it is to adjust the curve equation while procuring to not move bias scores up and/or match variance scores more, method in which a consistent variance -which is the case with the majority of variance scores being 25%-, is a desirable characteristic, as moving the curve should not have further complications other than a simple adjustment.

Both models present low bias and high variance, with *XGBoost* having a more consistent variance throughout features than the Logistic Regression + Gradient Descent learning model.

An overall summary is shown below:

**Table 11.** Evaluation metrics.

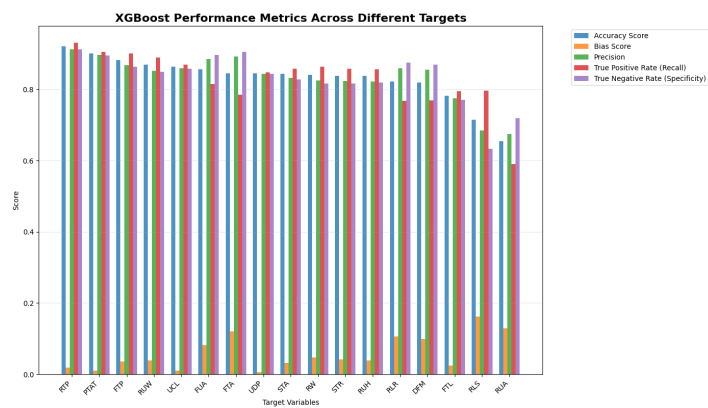
| Metric                            | scores |
|-----------------------------------|--------|
| Total true positives (TP):        | 2684   |
| Total true negatives (TN):        | 2716   |
| Total false positives (FP):       | 542    |
| Total false negatives (FN):       | 552    |
| Overall bias score (Average):     | 0.059  |
| Overall variance score (Average): | 0.249  |

**Table 12.** Top 5 best performing models by accuracy score.

| No. | Feature | Accuracy | Bias | Precision |
|-----|---------|----------|------|-----------|
| 1.  | RTP:    | 92.1%    | 1.9% | 91.2%     |
| 2.  | PTAT:   | 90.1%    | 1.0% | 89.6%     |
| 3.  | FTP:    | 88.2%    | 3.7% | 86.9%     |
| 4.  | RUW:    | 86.9%    | 3.9% | 85.3%     |
| 5.  | UCL:    | 86.4%    | 1.0% | 86.0%     |



**(a)** Logistic regression performance.



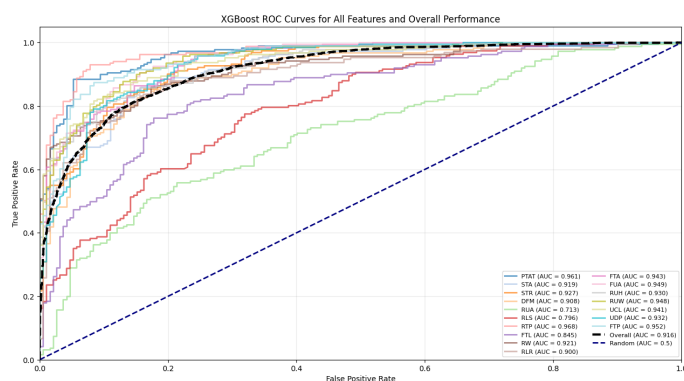
**(b) XGBoost performance.**

**Figure 14.** comparison between performance metrics for Logistic Regression + Gradient Descent and XGBoost.

**Table 13.** Top 5 best performing models by bias score.

| No. | Feature | Accuracy | Bias | Precision |
|-----|---------|----------|------|-----------|
| 1.  | UDP     | 84.6%    | 0.5% | 84.4%     |
| 2.  | PTAT:   | 90.1%    | 1.0% | 89.6%     |
| 3.  | UCL:    | 86.4%    | 1.0% | 86.0%     |
| 4.  | RTP:    | 92.1%    | 1.9% | 91.2%     |
| 5.  | FTL:    | 78.3%    | 2.4% | 77.4%     |

The *XGBoost* classification model has a more stable variance than the Logistic Regression + Gradient Descent classification model thanks to its calculation for the most efficient epoch number for all the features, but has a slightly higher bias as a result of the overfitting fix. Both models maintain an average variance of 24%, curve which can be adjusted, however, because the *XGBoost* model presents more consistent variance across features, adjusting the equation might be easier for this model than for the Logistic Regression + Gradient Descent model.



**Figure 15.** ROC curve for the XGBoost classification model.

*XGBoost* significantly outperforms logistic regression across almost all features, with most AUC scores above 0.9. Unlike the Logistic Regression + Gradient Descent model, *XGBoost* shows a consistently high performance across most features. Some features like ATP (AUC = 0.968), FYR (AUC = 0.961), and RIM (AUC = 0.949) show exceptional discriminatory power, and even though the overall AUC of 0.916 is slightly lower than logistic regression’s overall, it should be considered that *XGBoost* is already performing near-ceiling for individual features.

An Ensemble approach could be good for both models for their individual strengths by leveraging both Logistic Regression + Gradient Descent and *XGBoost*, as the strong performance suggests the data

set's features are well-constructed, but there may be opportunities to create even better features based on the patterns observed.

Now, comparing the confusion matrixes for both models (Fig. 8 and Fig. 11), it's visible that the *XGBoost* classification model has slightly higher misclassification rate (1094) than the Logistic Regression + Gradient Descent (1083) classification model.

However, by comparing each of the two training progresses numerically

**Table 14.** Overall metrics for Logistic Regression + Gradient Descent model.

|                            |        |
|----------------------------|--------|
| Final Training Loss:       | 0.2967 |
| Final Validation Loss:     | 0.2958 |
| Final Training Accuracy:   | 0.8852 |
| Final Validation Accuracy: | 0.8961 |

**Table 15.** Overall metrics for XGBoost model (after reducing overfitting).

|                            |        |
|----------------------------|--------|
| Final Training Loss:       | 0.4560 |
| Final Validation Loss:     | 0.4688 |
| Final Training Accuracy:   | 0.8245 |
| Final Validation Accuracy: | 0.8105 |

**Note:**

The reason of comparing training performance numerically with the final values rather than graphically, is that the *XGBoost* library calculates the most efficient training by epochs, meaning the training will stop as soon as no significant loss across epochs is seen, making our epochs per feature different values, meaning different  $X$  axis scales, and therefore, making it difficult to have a faithful graph with a fixed  $X$  axis for the whole training progress.

It's evident that the Logistic Regression + Gradient Descent learning model presented less overfitting than the *XGBoost* learning model with our sire data set, so the only way to discriminate between the two models result wise, is to meticulously tune the curve's equation to see which model has the best scores, but for now, seeing that both models have overall similar scores.

So even if the Logistic Regression + Gradient Descent classification model has less overfitting and better loss to accuracy ratio, it is important to consider *XGBoost* generally provides superior performance across features, as *XGBoost*'s optimization for structured/tabular data, and therefore, larger data sets with this format; while the Logistic



Regression + Gradient Descent classification model takes more CPU and RAM while training and validating.

### 13.3. Additional considerations: Softmax V.S Softpro functions

While using XGBoost, for assigning classes to new sires, *Softmax* looks for exact matches in outputting the probability for each class, while *softprob* looks for the closest most probable match for a class id. And when working with animals there's a high probability there's not going to be exact matches available (Brownlee, 2024), so a model's evaluation for these specific kind of situation should not depend on *softmax* for class matching in an independent breeding program for a house level production or a small industrial production where high quality semen is not contemplated.

## 14. Conclusion

The development of a logistic regression and gradient descent model made to classify the lineups/designations of sires based on 17 features provided insights into the strengths and limitations of the model compared to *XGBoost*, specially when it comes to predicting. The results indicate a very predictive dataset, with XGBoost generally providing superior performance across most features, but with less loss to accuracy ratio than the Logistic Regression + Gradient Descent classification model. For both models, even if overfitting was not fully reduced, it is enough for classifying a common sire if a farmer wants to have an insight on which lineup/designation it might belong, specially if a breeding program without resorting to buying semen is desired (a common preference among the majority of livestock projects in Mexico).

## References

- [1] GeeksforGeeks. (2025, September 5). *XGBoost*. GeeksforGeeks. <https://www.geeksforgeeks.org/machine-learning/xgboost/>
- [2] Holstein Association USA. (2024). *The world's largest dairy cattle breed association*. <https://www.holsteinusa.com/>
- [3] Holstein Canada. (2015). *Services - classification*. [https://www.holstein.ca/Public/en/Services/Classification/Breakdown\\_of\\_Training](https://www.holstein.ca/Public/en/Services/Classification/Breakdown_of_Training)
- [4] Huang, Z., Zhu, X., Ding, M., & Zhang, X. (2020). Medical image classification using a Light-Weighted hybrid neural network based on PCANet and DenseNet. *IEEE Access*, 8, 24697–24712. <https://doi.org/10.1109/access.2020.2971225>
- [5] Brownlee, J. (2024). Configure XGBoost “multi:softprob” Objective. *XGBoosting*. <https://xgboosting.com/configure-xgboost-multi-softprob-objective/>
- [6] Scikit-learn. (2025). *OneVsRestClassifier*. <https://scikit-learn.org/stable/modules/generated/sklearn.multiclass.OneVsRestClassifier.html>
- [7] Psaltakis, G., Rogdakis, K., Loizos, M., & Kymakis, E. (2024). One-vs-One, One-vs-Rest, and a novel Outcome-Driven One-vs-One binary classifiers enabled by optoelectronic memristors towards overcoming hardware limitations in multiclass classification. *Discover Materials*, 4(1). <https://doi.org/10.1007/s43939-024-00077-7>
- [8] Select Sires. (2024). *Default*. <https://www.selectsires.com/home>
- [9] Tayo, B. O. (2022). The difference between L1 and L2 regularization. *KDnuggets*. <https://www.kdnuggets.com/2022/08/difference-l1-l2-regularization.html>